

Interpreting Super-Resolution CNNs for Sub-Pixel Motion Compensation in Video Coding

Luka Murn
BBC Research & Development
London, United Kingdom
Dublin City University
Dublin, Ireland
luka.murn@bbc.co.uk

Alan F. Smeaton
Dublin City University
Dublin, Ireland
alan.smeaton@dcu.ie

Marta Mrak
BBC Research & Development
London, United Kingdom
marta.mrak@bbc.co.uk

ABSTRACT

Machine learning approaches for more efficient video compression have been developed thanks to breakthroughs in deep learning. However, they typically bring coding improvements at the cost of significant increases in computational complexity, making them largely unsuitable for practical applications. In this paper, we present open-source software for convolutional neural network-based solutions which improve the interpolation of reference samples needed for fractional precision motion compensation. Contrary to previous efforts, the networks are fully linear, allowing them to be interpreted, with a full interpolation filter set derived from trained models, making it simple to integrate in conventional video coding schemes. When implemented in the context of the state-of-the-art Versatile Video Coding (VVC) test model, the complexity of the learned interpolation schemes is significantly reduced compared to the interpolation with full neural networks, while achieving notable coding efficiency improvements on lower resolution video sequences. The open-source software package is available at <https://github.com/bbc/cnn-fractional-motion-compensation> under the 3-clause BSD license.

CCS CONCEPTS

• **Computing methodologies** → **Neural networks**; *Knowledge representation and reasoning*; *Computer vision problems*; **Learning linear models**; • **Software and its engineering** → **Open source model**; • **Theory of computation** → *Data compression*.

KEYWORDS

video compression, motion compensation, interpolation, machine learning, complexity reduction

ACM Reference Format:

Luka Murn, Alan F. Smeaton, and Marta Mrak. 2021. Interpreting Super-Resolution CNNs for Sub-Pixel Motion Compensation in Video Coding. In *Proceedings of the 29th ACM International Conference on Multimedia (MM '21), October 20–24, 2021, Virtual Event, China*. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3474085.3478326>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MM '21, October 20–24, 2021, Virtual Event, China

© 2021 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-8651-7/21/10...\$15.00
<https://doi.org/10.1145/3474085.3478326>

1 INTRODUCTION

Machine Learning (ML) approaches have demonstrated superior performance compared to traditional methods when applied to a variety of challenging computer vision and image processing tasks [5]. These developments have led to considerable research efforts focused on ways to integrate ML solutions into next generation video coding schemes. Tools based on Convolutional Neural Networks (CNNs) are increasingly deployed in various newly proposed video compression approaches [9], but they typically introduce a significant increase in coding complexity, making them largely unsuitable for inclusion in practical applications. Moreover, learning-based methods of manipulating input data need to be properly explained and understood, to mitigate potential unexpected outcomes.

Several tools based on simplified forms of ML have been adopted into the latest MPEG and ITU standard Versatile Video Coding (VVC) [3]. These carefully designed learning-based tools, such as the Low-Frequency Non-Separable Transform (LFNST) [8] and Matrix Intra-Prediction (MIP) [6], contribute towards VVC achieving considerably higher compression efficiency compared to its predecessor, High Efficiency Video Coding (HEVC).

Modern video compression formats rely on a block-based hybrid coding scheme, where each video frame is partitioned into rectangular blocks and encoded using predictive coding. The predictive coding unit within a video codec removes a video's redundancy in the spatial (intra-prediction) and temporal (inter-prediction) domains. When applying inter-prediction to obtain a prediction of the to-be-coded block, a block-matching algorithm is utilised to find the appropriate reference block in a previously encoded frame. Motion Compensation (MC) calculates the difference between the current block and the reference, also known as the residual. The prediction can be further refined using interpolation filtering to provide more accurate prediction samples, known as sub-pixel interpolation. Sub-pixel MC simulates frame-to-frame displacements.

As the process of super-resolution is an interpolation operation, super-resolution CNNs have been successfully adapted to produce new, efficient interpolation filters. However, the majority of proposed approaches were implemented in HEVC and have very high computational requirements [10, 13], increasing the encoding complexity up to 380 times when tested in a CPU environment [12].

This paper presents an approach based on interpreting trained linear super-resolution networks to derive a low-complexity implementation of ML interpolation filters [11, 12]. This is the first approach that shows coding improvements in the challenging VVC framework, while significantly reducing computational requirements.

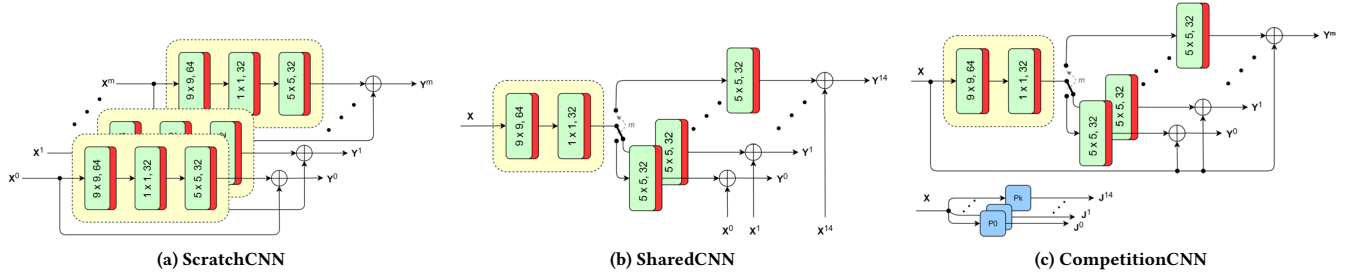


Figure 1: Quarter-pixel interpolation filtering with three linear network architectures. Each convolutional kernel size is indicated as (kernel height $\times$ kernel width, number of kernels)

The paper is structured as follows: Section 2 introduces CNN-based sub-pixel MC solutions, Section 3 specifies the evaluation methods, with conclusions drawn in Section 4.

2 CNN-BASED SUB-PIXEL MOTION COMPENSATION

Modern video coding typically utilises a set of fixed interpolation filters to refine prediction samples obtained by inter-prediction MC. In practical solutions, these are designed to interpolate samples at a half-pixel or a quarter-pixel level. Advanced video coding solutions can employ multiple filter sets to better generalise across diverse video content. Besides common filters based on Discrete Cosine Transforms (DCT), VVC introduces an alternate half-pixel interpolation filter and a separate filter set for Affine motion [3].

Several learning-based approaches have been proposed that use CNNs to generate additional sub-pixel interpolation filter sets. These solutions integrate the trained network as an alternate, switchable interpolation filter set within a codec, giving it the option to select between traditional and CNN-based filters when encoding the luma channel of a partitioned block within a frame. Contrary to efforts which incorporate the entire neural network model within HEVC which can result in increasing the decoding time 700 times on average [10, 13], the techniques presented in [11, 12] rely on interpreting trained linear networks to achieve a low-complexity implementation of CNN models within VVC.

2.1 ScratchCNN

In [12], a model named ScratchCNN, illustrated in Fig. 1a, is obtained by removing non-linearities from the Super-Resolution CNN (SRCNN) architecture [4], with no padding between network layers, so the input size is reduced from one layer to the next. For a given reference block, or input X^m , the ScratchCNN network predicts residuals R^m , where $m = 0, \dots, 14$ corresponds to each of 15 half and quarter pixel positions. Each residual is then added to the input reference samples to form the prediction Y^m representing values at a quarter-pixel position as $Y^m = R^m + X^m$.

ScratchCNN is a linear model, so it can be expressed as a series of matrix operations from the input to the output. The process of acquiring an interpolated sample from the input area is illustrated in Fig. 2. Due to the convolutional kernel dimensions (9×9 , 1×1 , 5×5), an input area of 13×13 is required to compute the output.

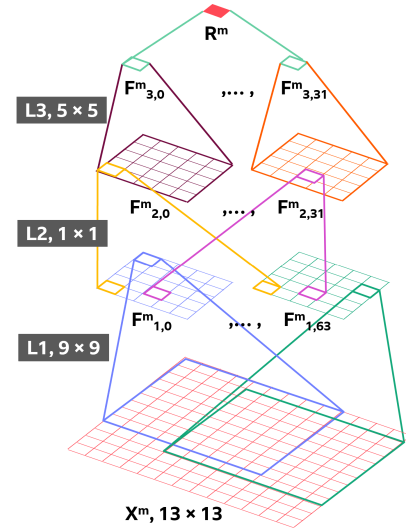


Figure 2: Fractional pixel derivation process for branch m of an NN interpolation filter set.

As illustrated in Fig. 2, the first layer, L1, extracts features from regions of input area X^m by applying 9×9 convolutional kernels, denoted as $K^m_{1,i}$, to obtain the first layer output, F^m_1 . It is defined as:

$$F^m_{1,i} = K^m_{1,i} * X^m, \quad (1)$$

where $i = 0, \dots, 63$. The second layer, L2, combines features extracted from the same regions to retrieve its output, F^m_2 , as:

$$F^m_{2,j} = \sum_{i=0}^{63} K^m_{2,j} F^m_{1,i}, \quad (2)$$

where $K^m_{2,j}$ corresponds to 1×1 convolutional kernels and $j = 0, \dots, 31$. The network then combines all extracted features to reconstruct the pixel R^m from F^m_3 feature maps comprising of 5×5 convolutional kernels $K^m_{3,j}$ as:

$$F^m_{3,j} = K^m_{3,j} * F^m_{2,j}, \quad (3)$$

and their summation for each $j = 0, \dots, 31$ as:

$$\mathbf{R}^m = \sum_{j=0}^{31} \mathbf{F}_{3,j}^m. \quad (4)$$

Lastly, the required, non-separable 2D interpolation filter set $\mathbf{M}^m$ can be obtained from the series of convolutional kernel weights $\mathbf{K}_{1,2,3}^m$ which were multiplied with values in the input $\mathbf{X}^m$. The interpolated sample can be described as:

$$\mathbf{R}^m = \mathbf{M}^m * \mathbf{X}^m = \mathcal{F}(\mathbf{K}_3^m, \mathbf{K}_2^m, \mathbf{K}_1^m, \mathbf{X}^m). \quad (5)$$

The coefficients of each filter $\mathbf{M}^m$ represent the contribution of reference samples towards the final prediction in a 13×13 surrounding area. The filter set corresponds to the 15 possible quarter-pixel fractional positions, each trained with 15 ($m = 0, \dots, 14$) separate sets of samples $\mathbf{X}^m$ already classified before training.

2.2 SharedCNN

In [11], an architecture called SharedCNN is proposed where a so-called trunk, comprising of the first two layers in the network, is shared among all branches which still correspond to individual sub-pixel positions. This is illustrated in Fig. 1b. The branches are trained for only one sub-pixel position, and the trunk is trained for all positions. This solution is an improvement over ScratchCNN as only one network needs to be trained, and the trunk allows the branches to share information amongst themselves, as they are trying to approximate a very similar problem. The equations describing the SharedCNN network are equivalent to Eq. 1 - 5, with the first two convolutional layers defined as same for all sub-pixel positions:

$$\mathbf{K}_1 = \mathbf{K}_1^m, \mathbf{K}_2 = \mathbf{K}_2^m, \forall m = 0, \dots, 14. \quad (6)$$

From the inference perspective, shared layers are combined with each individual branch to produce a non-separable 2D filter for each fractional location, as in ScratchCNN.

2.3 CompetitionCNN

In [11], a new training framework is defined, on top of the architecture with shared layers in Fig. 1b, by minimizing the loss of each prediction from all neural network filters, as well as standard filters (P_0 to P_{14}). The network is named CompetitionCNN and is detailed in Fig. 1c.

The input-output pairs for training the proposed approaches are formed by VVC. The ground truth is the uncompressed rectangular block in the current frame, while the input to the network is the decoded reference block that was chosen for sub-pixel interpolation. An important aspect of the generated pairs that needs to be preserved is the sub-pixel shift m from the reference input block to the output. The resulting dataset D contains subsets d^m and their relationship is described as $D = \bigcup d^m$.

Due to its architecture constraint, each ScratchCNN model as well as SharedCNN had to be trained with a specific subset d^m , consisting of pairs $(\mathbf{X}^m, \mathbf{Y}^m)$. However, the CompetitionCNN relaxes this fragmentation, using the whole dataset D for all branches.

CompetitionCNN utilises a three-stage training framework. In the first training epoch, all layers of the network are updated jointly with the loss of the whole dataset D in all branches. During the

second epoch, each branch b of the last layer of the neural network is pre-trained with a separate subset d^m of the dataset, such that $m = b$, equivalent to each quarter-pixel position. By applying this step, a higher correlation between a branch output and a distinct quarter-pixel filter is established. Lastly, after the two pre-training steps are finished, training is commenced with a loss that updates the two shared layers and only the branch of the network which minimizes the loss for particular blocks, if the prediction it produces has lower loss than any of the standard predictions $\mathbf{J}^m$, $m = 0, \dots, 14$, obtained by corresponding standard filters which are denoted as P_m , Fig. 1c. This stage ensures that the network competes with the traditional VVC filters during the network training process already, leading to a closer representation of its final implementation form – a switchable interpolation filter set.

3 EVALUATION

The VVC Test Model (VTM) [7] version 6.0 was used as basis for our implementation. Common Test Conditions (CTCs) defined by JVET [2] were used. CTC video sequence classes are defined as sequences of the same spatial resolution.

3.1 Data generation and training configuration

The training dataset for the proposed approach of fractional interpolation using CNNs is created by encoding the CTC video sequence *BlowingBubbles* under the Random Access (RA) configuration. In order to generate accurate training data, a set of restrictions has been applied to the CTC conditions. The flags are: Triangle=0, Affine=0, DMVR=0, BIO=0, WeightedPredP=0, WeightedPredB=0. ScratchCNN requires four sets (for Quantisation Parameters, i.e. QPs, 22, 27, 32 and 37) of 15 networks to be trained, one for each of the fractional positions. SharedCNN and CompetitionCNN simplify this process by requiring training of only one network and only for QP 27. The networks were trained for 1000 epochs with a batch size of 32, using an Adam optimizer and a learning rate of 0.0001.

3.2 Integration into VVC

After training the networks and extracting corresponding simplified 13×13 filter matrices from learned models, the filter set was integrated within the VTM encoder as switchable interpolation filters. The selection between the conventional VVC filters and the NN filters is performed at a Coding Unit (CU) level. One additional flag is correspondingly encoded in the bitstream and parsed by the decoder to determine which filter is used on a given block. Blocks coded in merge mode inherit usage of the same filter together with the merged motion information. The NN filters are only used for the luma component. The same encoding configuration flags were used as for generating the dataset, along with these additional restrictions: MHIntra=0, SBT=0, MMVD=0, SMVD=0, IMV=0, SubPuMvp=0, TMVPMODE=0.

3.3 Compression performance assessment

Tests were performed for simplified VVC inter-prediction conditions, to resemble those of HEVC, with all reported results obtained in a CPU environment. The interpolation filter set extracted from

Table 1: Comparison of coding complexity for linear and non-linear network structures for Class D sequences, LDB configuration, 32 frames.

	Encoding Time [%]	Decoding Time [%]
SRCNN	38915%	1322%
ScratchCNN	807%	177%

a trained model and implemented in a switchable manner is compared to the anchor that does not contain this extension, but retains the same encoding restrictions.

To evaluate whether linear models that can be condensed into 13×13 matrices reduce coding complexity compared to full neural networks when integrated into VVC as switchable interpolation filters, a test was performed using trained SRCNN and ScratchCNN networks. All weights and biases (8129 parameters) are extracted from trained SRCNNs and implemented as a series of matrix multiplications. In contrast, each trained ScratchCNN model is condensed in one 2D matrix that contains 169 parameters. As presented in Table 1 which summarises results for coding 32 frames of Class D sequences under the low-delay B (LDB) configuration, the encoding time for a CPU implementation of SRCNN equals to 38915% of the equivalent VVC anchor. ScratchCNN encoding time increases are 807%, showing a considerable running time reduction compared to SRCNN.

The ScratchCNN architecture requires different networks to be trained for each fractional shift. With SharedCNN and CompetitionCNN, a single network can be trained to approximate all possible fractional shifts because of the inclusion of a two-layer trunk shared across all network branches. As presented in Table 2, when tested as switchable filter sets within VVC on 32 frames, both SharedCNN and CompetitionCNN exhibit higher Bjøntegaard delta-rate (BD-rate) [1] savings, especially under the low-delay P (LDP) configuration, compared to the previous ScratchCNN approach. Another significant improvement introduced with the CompetitionCNN network is the competition of the branches amongst themselves when trained on a dataset.

When tested on all frames of lower resolution CTC sequences, CompetitionCNN displays coding gains under all encoding configurations, reaching 2.58% savings for Class D sequences in LDP, as stated in Table 3.

4 CONCLUSIONS

In this paper, an approach for developing sub-pixel interpolation filters from linear CNNs was presented. The linear network structure allows layers to collapse into a 2D matrix. In this way, it is possible to directly extract trained filter coefficients and integrate them into a video codec as switchable interpolation filters. Experimental results show a considerable decrease in encoder and decoder running time when compared to previous state-of-the-art methods, with notable coding savings on low-resolution sequences in a modified VVC encoder environment, enabling possible practical implementations of NN-based interpolation filters within video codecs.

Table 2: BD-rate luma comparison of different network structures on 32 frames for Class D sequences.

Model	Encoder configuration		
	RA	LDB	LDP
ScratchCNN	-0.26%	-0.58%	-1.11%
SharedCNN	-0.34%	-0.53%	-1.21%
CompetitionCNN	-0.62%	-1.06%	-2.21%

Table 3: Coding performance of CompetitionCNN for RA, LDB and LDP configurations; BD-rate for luma; tested on all frames of CTC sequences

Class	Encoder configuration		
	RA	LDB	LDP
Class C	-0.73%	-1.07%	-1.92%
Class D	-0.80%	-1.48%	-2.58%

ACKNOWLEDGMENTS

This work was conducted within the project JOLT funded by the European Union's Horizon 2020 research and innovation programme under Marie Skłodowska Curie grant agreement No 765140.

REFERENCES

- [1] G. Bjøntegaard. 2001. Calculation of average PSNR differences between RD-curves. *VCEG-M33* (2001).
- [2] J. Boyce, K. Suehring, X. Li, and V. Seregin. 2018. JVET Common Test Conditions and Software Reference Configurations. *Document JVET-J1010* (2018).
- [3] B. Bross, J. Chen, J. R. Ohm, G. J. Sullivan, and Y. K. Wang. 2021. Developments in International Video Coding Standardization After AVC, With an Overview of Versatile Video Coding (VVC). *Proc. IEEE* (2021), 1–31. <https://doi.org/10.1109/JPROC.2020.3043399>
- [4] C. Dong, C. C. Loy, K. He, and X. Tang. 2016. Image Super-Resolution Using Deep Convolutional Networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 38, 2 (2016), 295–307. <https://doi.org/10.1109/TPAMI.2015.2439281>
- [5] Y. Guo, Y. Liu, A. Oerlemans, S.-Y. Lao, S. Wu, and M. Lew. 2015. Deep Learning for Visual Understanding: A Review. *Neurocomputing* 187 (11 2015). <https://doi.org/10.1016/j.neucom.2015.09.116>
- [6] P. Helle, J. Pfaff, M. Schäfer, R. Rischke, H. Schwarz, D. Marpe, and T. Wiegand. 2019. Intra Picture Prediction for Video Coding with Neural Networks. In *Data Compression Conference (DCC)*. 448–457. <https://doi.org/10.1109/DCC.2019.00053>
- [7] S. Kim J. Chen, Y. Ye. 2019. Algorithm Description for Versatile Video Coding and Test Model 6 (VTM). *Document JVET-O2002* (2019).
- [8] M. Koo, M. Salehifar, J. Lim, and S-H Kim. 2019. Low Frequency Non-Separable Transform (LFNST). In *2019 Picture Coding Symposium (PCS)*. 1–5. <https://doi.org/10.1109/PCS48520.2019.8954507>
- [9] D. Liu, Y. Li, J. Lin, H. Li, and F. Wu. 2020. Deep Learning-Based Video Coding: A Review and a Case Study. *ACM Comput. Surv.* 53, 1 (2020). <https://doi.org/10.1145/3368405>
- [10] J. Liu, S. Xia, W. Yang, M. Li, and D. Liu. 2019. One-for-All: Grouped Variation Network-Based Fractional Interpolation in Video Coding. *IEEE Transactions on Image Processing* 28, 5 (2019), 2140–2151. <https://doi.org/10.1109/TIP.2018.2882923>
- [11] L. Murn, S.G. Blasi, A.F. Smeaton, and M. Mrak. 2021. Improved CNN-based Learning of Interpolation Filters for Low-Complexity Inter Prediction in Video Coding. *IEEE Open Journal of Signal Processing* (2021), 1–1. <https://doi.org/10.1109/OJSP.2021.3089439>
- [12] L. Murn, S.G. Blasi, A.F. Smeaton, N.E. O'Connor, and M. Mrak. 2020. Interpreting CNN For Low Complexity Learned Sub-Pixel Motion Compensation In Video Coding. In *IEEE International Conference on Image Processing, ICIP 2020, IEEE*, 798–802. <https://doi.org/10.1109/ICIP40778.2020.9191193>
- [13] N. Yan, D. Liu, H. Li, B. Li, L. Li, and F. Wu. 2019. Invertibility-Driven Interpolation Filter for Video Coding. *IEEE Transactions on Image Processing* 28, 10 (2019), 4912–4925. <https://doi.org/10.1109/TIP.2019.2913092>